M8 — What this block gets you

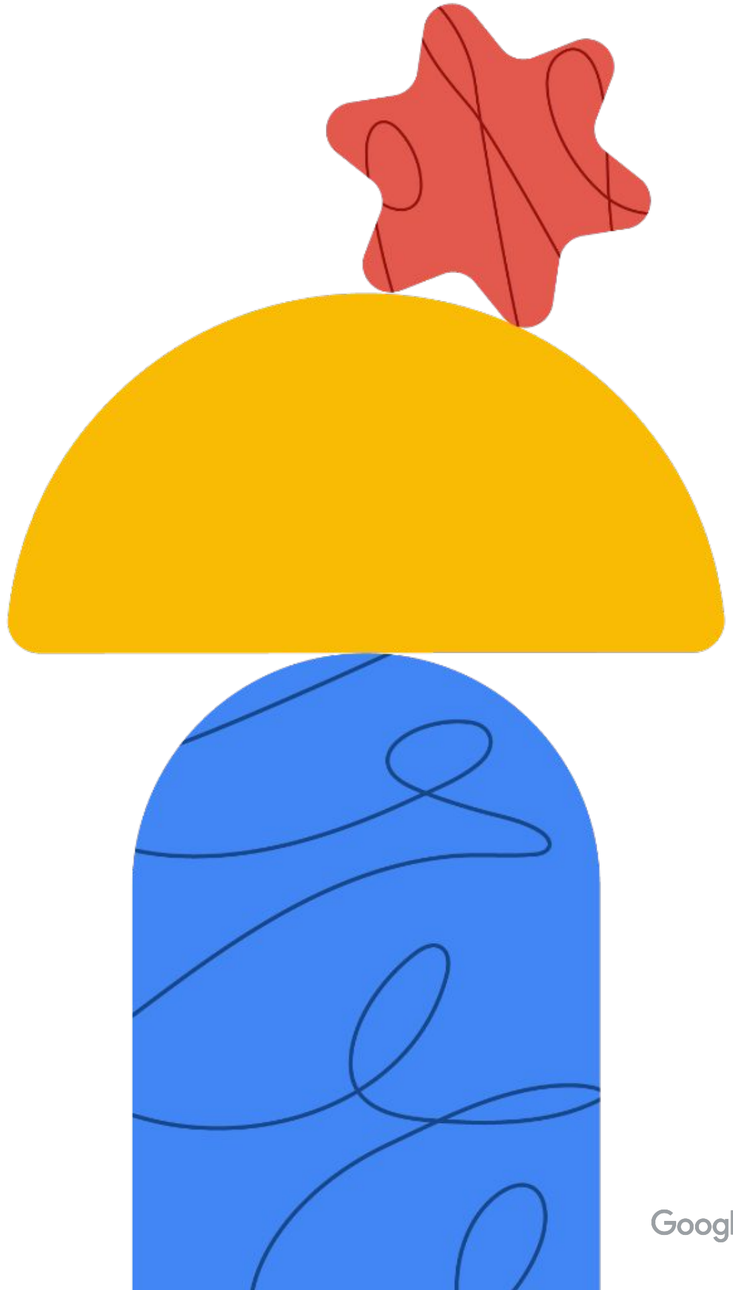
- GCS + open formats (Parquet, Iceberg) — the Bronze foundation in your own architecture doc
- Apache Iceberg — warehouse behavior (ACID, schema evolution, time travel) on raw files in a bucket
- BigQuery as the one SQL engine over Bronze, Silver and Gold — managed tables and lake files alike
- Federated queries — your linked-server / OPENQUERY pattern, from BigQuery, without moving data
- Hands-on: the federated-query lab, then external tables on Hive-partitioned Parquet — Bronze made tangible

This block and the hands-on morning

09:45–10:30	M8 — Building a data lakehouse: GCS, open formats, Iceberg, federated queries	
10:30–10:45	Break	
10:45–11:15	Lab: Federated Query with BigQuery — live operational data, no copy	LAB
11:15–11:35	DIN: Exploring external tables — Hive-partitioned Parquet on GCS	DIN
11:35–12:00	M9 — Modernizing data warehouses with BigQuery and BigLake	

Proprietary + Confidential

Data storage for data lakehouses on Google Cloud



Proprietary + Confidential

Data lake

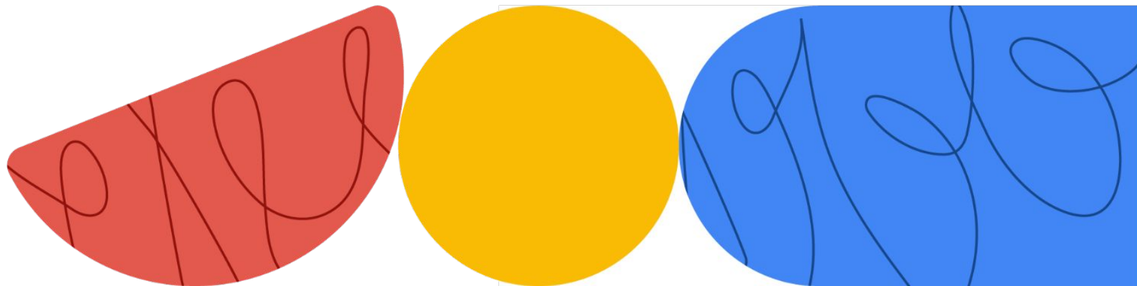
Store raw, unstructured, or structured data

Data warehouse

Schema enforcement and query performance

Data lakehouse

Offers a unified system to handle data needs



Proprietary + Confidential

Cloud Storage

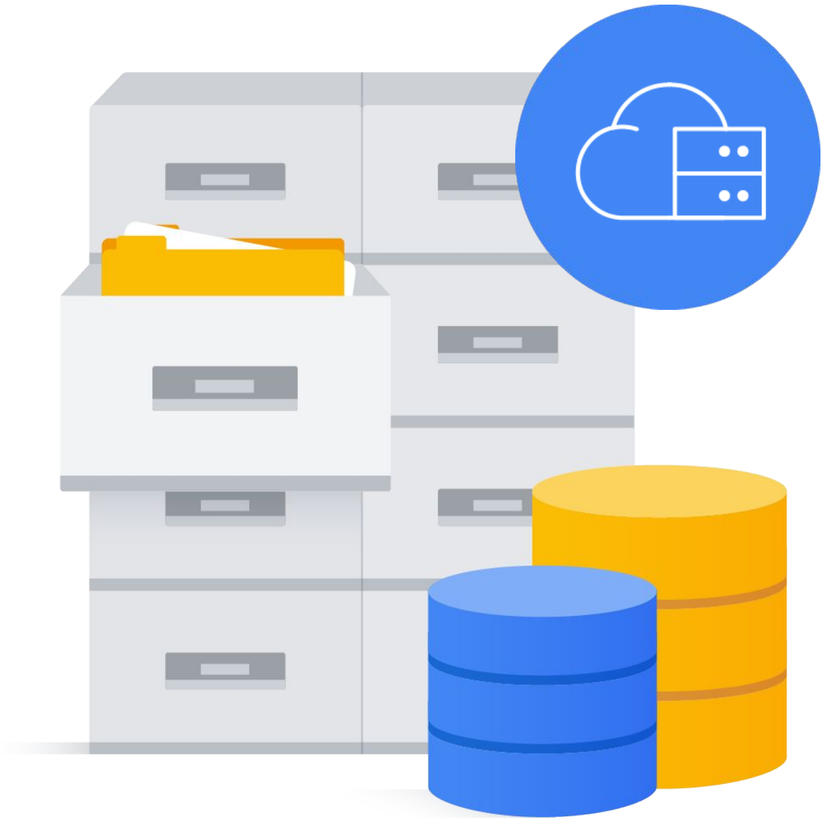
A massively scalable, highly durable, and cost-effective object storage service.



Proprietary + Confidential

Other storage options

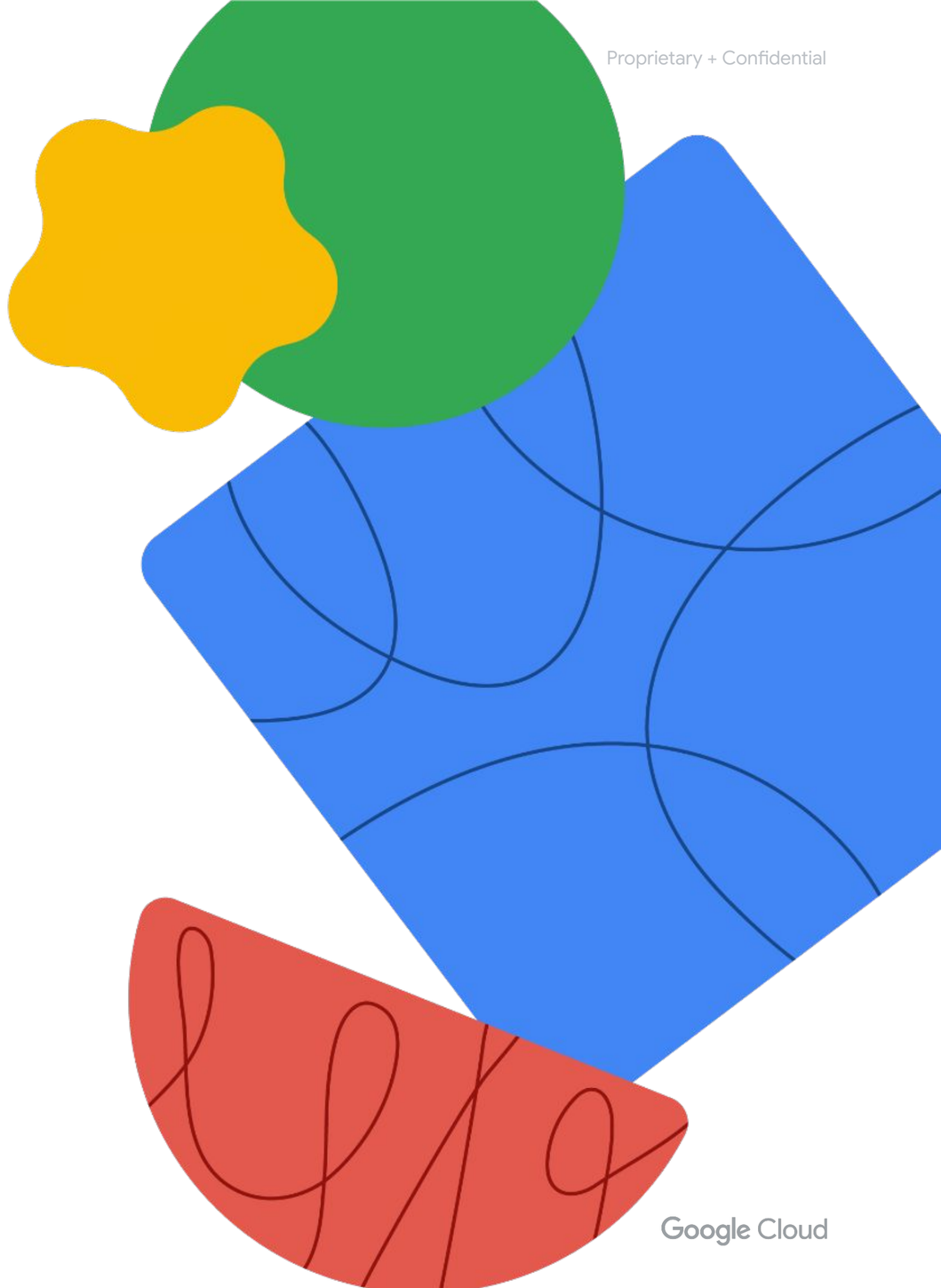
If your strategy is multicloud, you can keep your data in another major provider's cloud storage.



Proprietary + Confidential

Introduction to Apache Iceberg open table format

A data lakehouse needs structure and performance. Open table formats, such as Apache Iceberg provide this.



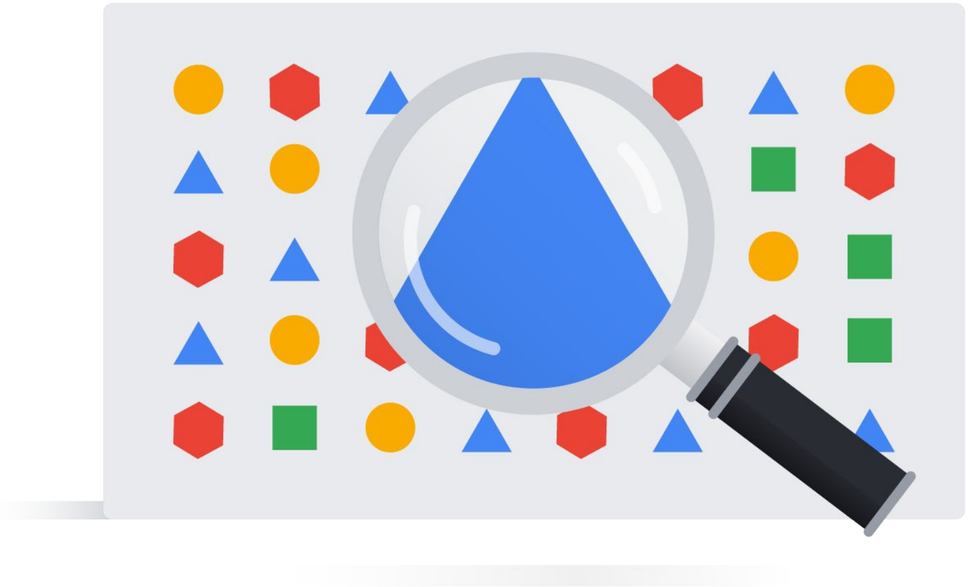
Proprietary + Confidential

Efficient querying with Apache Iceberg (1 of 2)

Cloud Storage

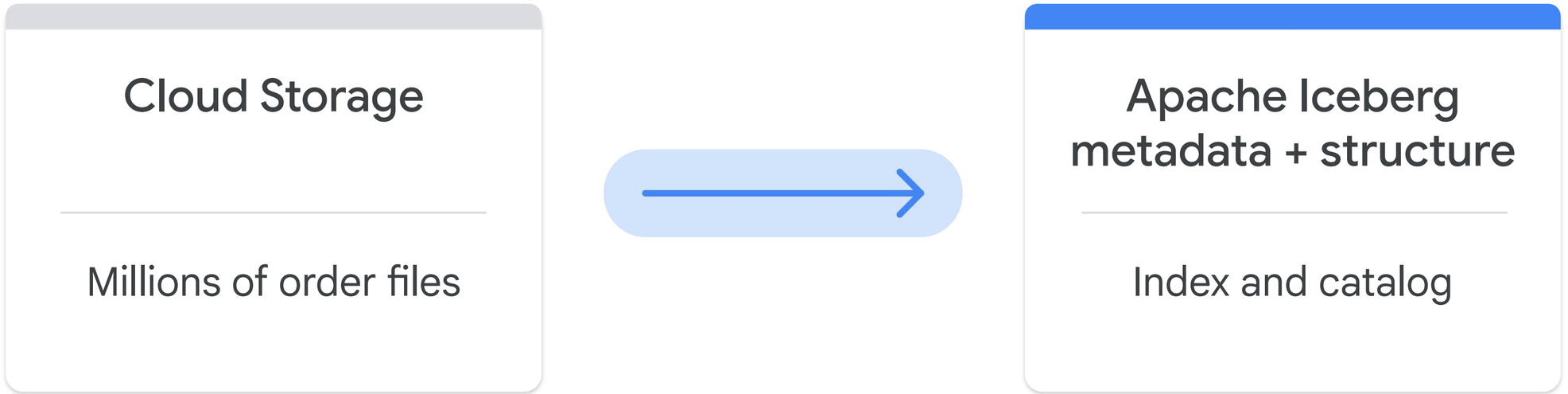
Millions of order files

Without Apache Iceberg: Query scans every file



Proprietary + Confidential

Efficient querying with Apache Iceberg (2 of 2)



Without Apache Iceberg: Query scans every file

With Apache Iceberg: Metadata directs queries to only relevant data

Proprietary + Confidential

What does Apache Iceberg provide?

01

Schema evolution

Allows schema changes (such as adding and renaming columns) without impacting existing data or applications.

02

Hidden partitioning

Automatically manages how data is partitioned for efficient querying.

03

Time travel

Enables querying past data versions for auditing and debugging, allowing users to reproduce exact data states from previous points in time.

04

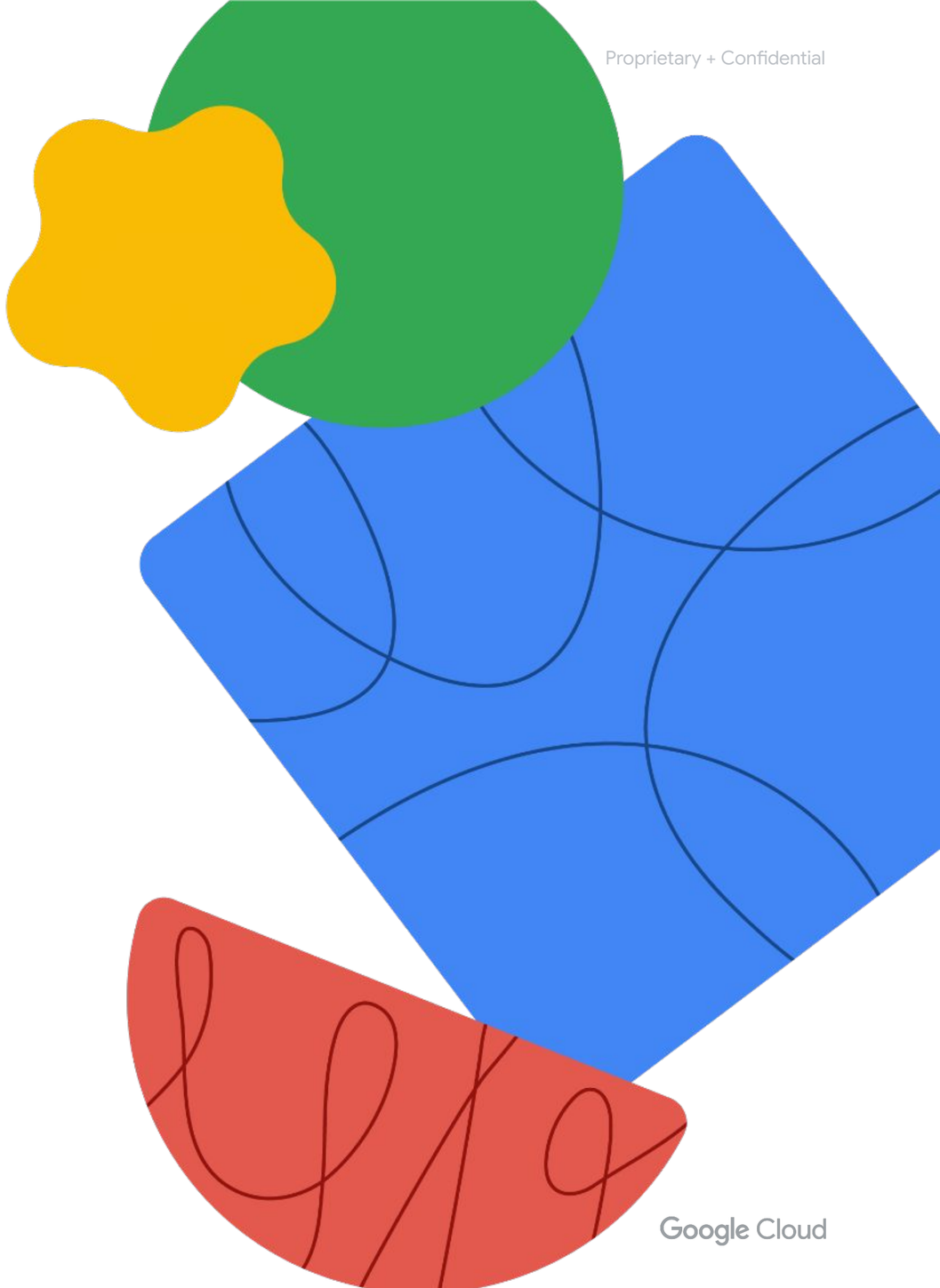
Atomic transactions

Enable reliable, concurrent data operations, preventing corruption when multiple processes read/write to the same tables.

Proprietary + Confidential

Using Apache Iceberg with Cloud Storage

Treat a collection of data files as performant, queryable tables, bringing the benefits of a data warehouse to a flexible data lake.



GCS + open formats = your Bronze foundation

- Today— county and vendor file drops sit unqueryable until an SSIS load runs
- Bronze target— Parquet files in a GCS bucket, Hive-partitioned by date, queryable immediately
- Iceberg on top— ACID, schema evolution, time travel over those same files, multi-engine readable
- Silver and Gold— BigQuery-managed tables, built by Dataform ELT from Bronze (this afternoon)
- One SQL surface— BigQuery reads managed tables and lake files alike; no staging hop
- AlloyDB— operational/OLTP sibling (managed PostgreSQL); not your path, one line only

Proprietary + Confidential



BigQuery as the central processing engine

Whereas Cloud Storage and Apache Iceberg provide a flexible, open-standard foundation for storing vast amounts of raw and structured data, BigQuery is the high-performance engine that activates it.

Proprietary + Confidential

BigQuery

Seamless data querying with BigQuery and Lakehouse

By creating Lakehouse tables, Cymbal can use BigQuery's familiar SQL interface to directly and securely query the Apache Iceberg-formatted data in their Cloud Storage data lake.

Eliminating data duplication and ETL processes

No need to duplicate data or perform costly extract, transform, load (ETL) processes to run analytics.

Enhanced analytics performance for data lake

Analysts get the performance and features of a premier data warehouse while querying data directly in their open-format data lake.

Proprietary + Confidential

BigQuery

Native storage for “hot” datasets

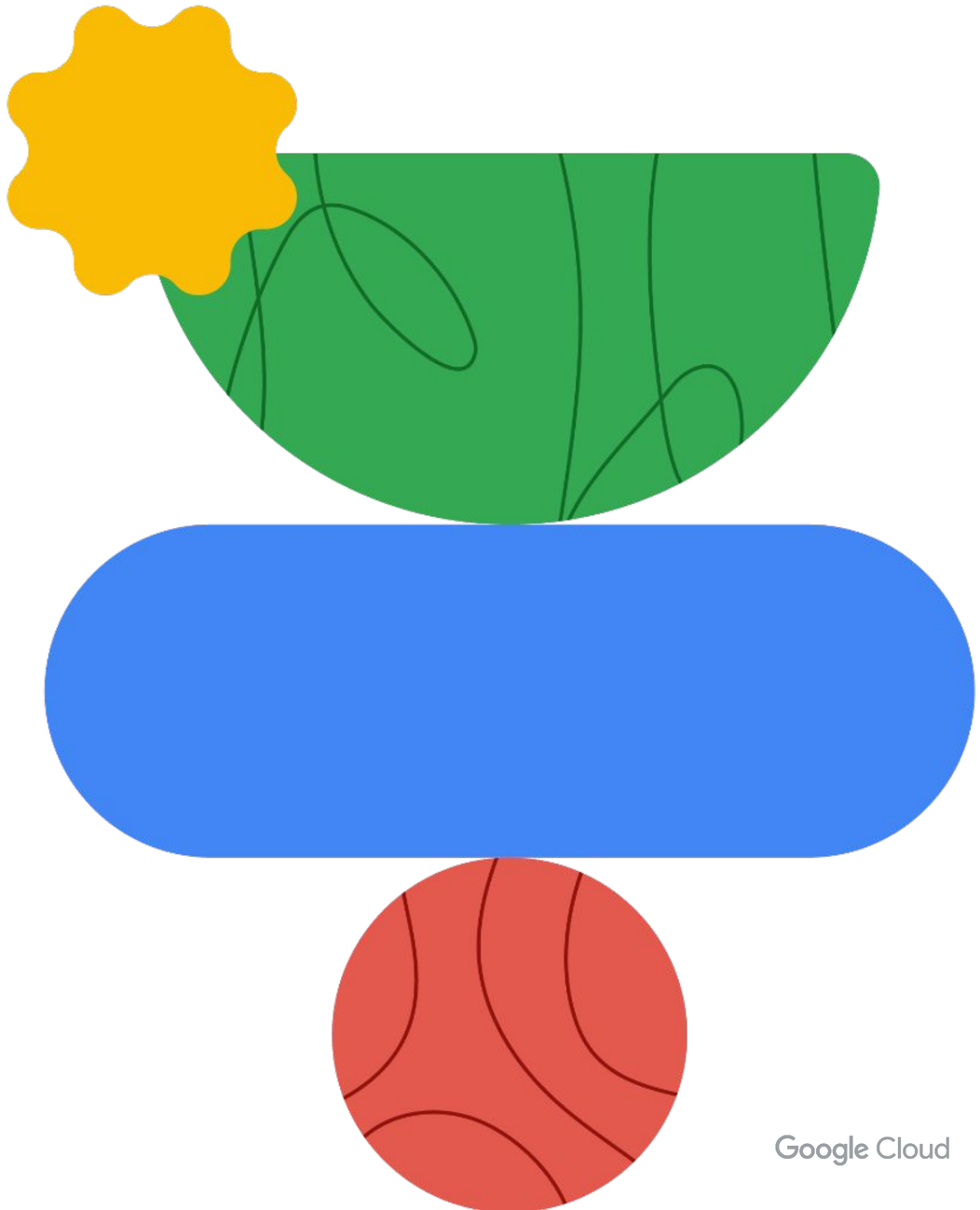
BigQuery offers optimized, managed storage for frequently accessed or "hot" datasets. This is ideal for data requiring the fastest query performance, such as key marketing performance dashboards.

Unified data analysis platform

Cymbal can leverage a unified platform to analyze data located in their data lake, BigQuery's native storage, or a combination of both, all accessible through a single interface.

Proprietary + Confidential

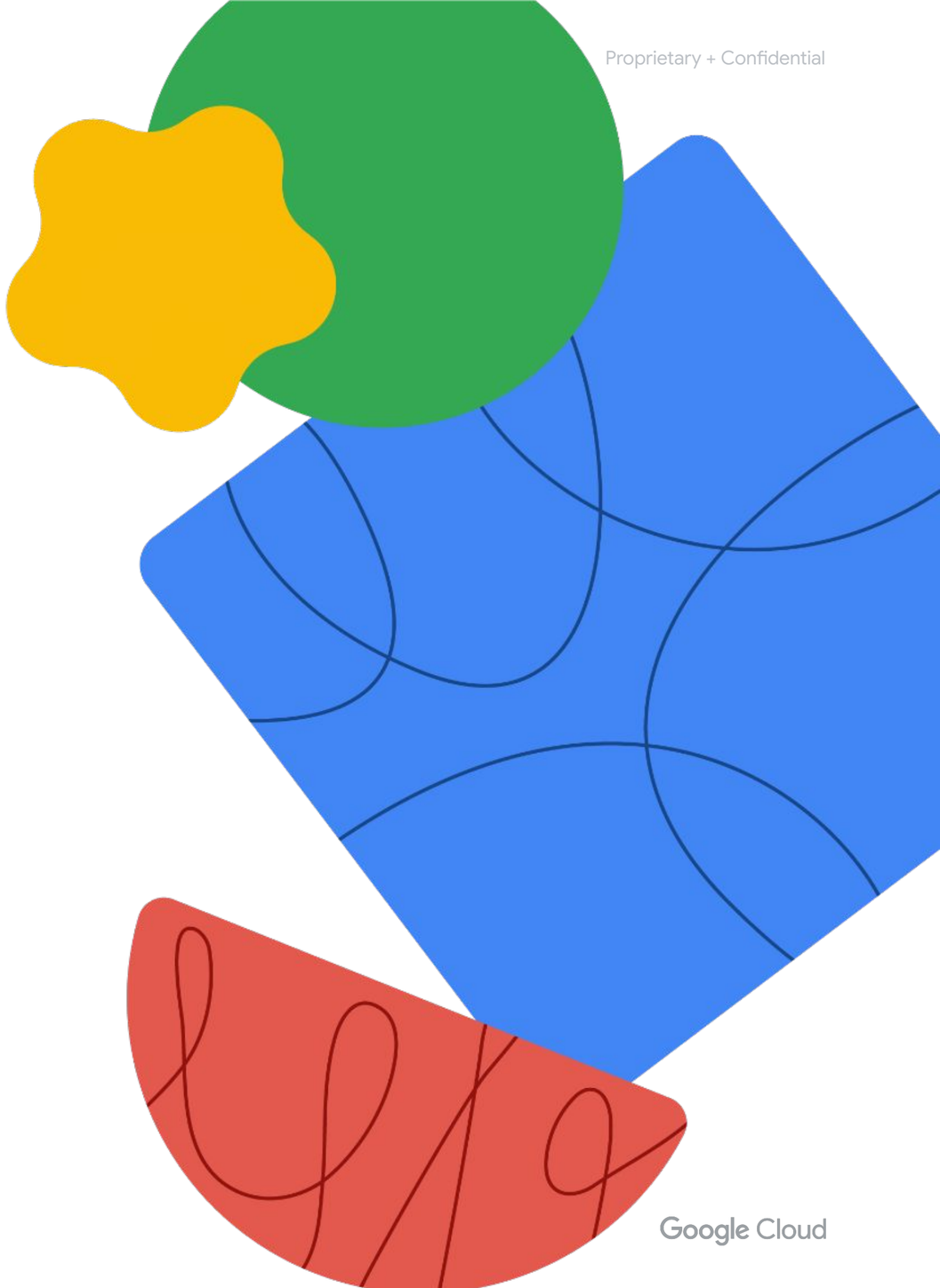
Combining operational data in AlloyDB



Proprietary + Confidential

Federated queries: Combining operational and analytical data in a single query

Query data in external systems, such as an operational AlloyDB database, in real time without moving or copying it into BigQuery.



Proprietary + Confidential

Combining data

AlloyDB real-time inventory data: Secure connection (one-time setup)

Cloud Storage + Apache Iceberg historical sales data



BigQuery unified analysis

Use `EXTERNAL_QUERY` (connection ID + SQL to join AlloyDB and Apache Iceberg data

Google Cloud

LAB

Lab: Federated Query with BigQuery

30 minutes · 10:45–11:15 · skills.google lab seat — pre-warmed before the break ends

1. Launch the lab; create the BigQuery connection resource — your linked-server definition, one-time setup
2. Run your first EXTERNAL_QUERY — the inner SQL is PostgreSQL on the AlloyDB side, just like OPENQUERY
3. Join historical analytical data in BigQuery with live transactional rows — one query, two systems
4. Debrief: querying data where it lives — that is the lakehouse promise

Why it matters — your SQL Server systems stay operational through the migration — federation means you query them in place instead of moving everything first.

Proprietary + Confidential

Lab: Federated query with BigQuery and AlloyDB

Overview

Explore how federated queries allows you to gain real-time insights by directly querying an operational database from within BigQuery.



What you'll do

- Connect BigQuery to an AlloyDB for PostgreSQL database.
- Join historical analytical data with live transactional data.

DIN · free

DIN: Exploring external tables

20 minutes · 11:15–11:35 · BigQuery console + GCS bucket jwd-gcp-demos/orders_partitioned

1. Browse the bucket — directories named order_date=YYYY-MM-DD, each holding Parquet files: Hive layout
2. Create external table ext_part in your class dataset — Parquet, source-data partitioning, auto-detect schema
3. Query the full table — every file scanned; note the bytes processed
4. Filter on order_num (a data column) — still a full scan; a WHERE clause alone saves nothing
5. Filter on order_date (the partition key) — bytes drop hard: partition pruning on raw files, no load job ran

Why it matters — Bronze made tangible — open-format files in a bucket, queryable in standard SQL with zero loading; delete the table and the files are untouched.

M8 — What you now have

- The Bronze blueprint— Parquet/Iceberg files in GCS, Hive-partitioned, queryable in place from BigQuery
- Iceberg's warehouse behavior on lake files— ACID, schema evolution, time travel — for the 13:00 lab
- Federation done hands-on— live operational data joined from BigQuery without an extract
- An external table over partitioned Parquet— you watched partition pruning cut bytes on raw files
- Next: M9 (11:35)— modernizing the warehouse itself: partitioning, clustering, BigLake